TRACK SELECTION & SOLUTION PROPOSAL

AI Revenue Recovery

Why Track 03 was selected for the Razorpay AI Buildathon, and the detailed design of the recovery agent we are building for it.

Selected track	03 — AI Revenue Recovery
Program	Razorpay AI Buildathon, student track
Deliverables	Working repository, 5-minute demo video, architecture notes

This document explains the reasoning behind the track choice and lays out the solution architecture in full, for review ahead of the build.

SECTION 1

Why We Selected Track 03

Five tracks were available: Agentic Commerce, AI Risk Manager, Revenue Recovery, Finance Controller, and an Open Track. Each was assessed against four criteria that matter most in a short, evidence-based build: how clearly the scope is bounded, how feasible it is to produce a defensible metric, how realistic the data requirement is, and how directly the result maps to Razorpay's actual product. Track 03 scored highest across all four, and is the only one where a full, working, well-tested submission is comfortably achievable in the time available.

How the tracks compare

Track	Main risk	Verdict
01 · Agentic Commerce	Brief is broad — “grow revenue” or “make a merchant transactable” leaves too much open to interpretation.	High ceiling, but easy to over-scope and run out of runway.
02 · AI Risk Manager	Requires genuine precision and recall on a held-out test set — difficult to report honestly without real labeled fraud data.	Metric integrity is hard to guarantee on synthetic data alone.
03 · Revenue Recovery	Needs disciplined stopping rules and a full audit trail — but the underlying loop is well-defined.	Selected. Full rationale below.
04 · Finance Controller	Needs a clean, large reconciliation dataset with genuine edge cases to be convincing.	Workable, but a more generic demo — reconciliation is a familiar shape.
05 · Open Track	No predefined scaffolding, and the same execution bar applies with none of the guardrails.	Highest risk of ambiguity consuming the build timeline.

The deciding factors

- **An explicit, hittable bar.** The brief asks for measured money recovered across a batch, with compliant escalation, stopping rules, and an audit trail — a concrete deliverable rather than a subjective judgment call.
- **Data we can legitimately synthesize.** Failed-payment batches — decline codes, amounts, timestamps — can be modeled realistically without needing proprietary fraud-labeled data.

- **A named example direction.** Both “failed-subscription recovery” and “payment degradation → root cause → recovery action” appear explicitly in the brief, signalling this is a shape the reviewers already recognize.
- **Direct relevance to Razorpay's business.** Payment recovery sits inside Razorpay's own payments and subscriptions stack, using their test-mode APIs — this is not a tangential idea, it is a feature they could plausibly ship.
- **Genuinely finishable.** The detect → decide → execute → audit loop is narrow enough to fully build, test, and polish, rather than leaving something half-built.

SECTION 2

The Solution

Failed Payment & Subscription Recovery Agent

Problem statement

Merchants using recurring billing and one-time payments lose revenue whenever a charge fails: a declined card, insufficient funds, an expired card, or a bank timeout. Today this is handled either with no follow-up or with blind, undifferentiated retries — both of which either waste recoverable revenue or annoy customers with retries that were never going to succeed. The agent's task is to close this loop end to end: detect the failure, diagnose why it happened, choose the correct intervention, execute it, and prove — with real numbers — how much revenue it recovered.

System architecture

The system is a five-stage pipeline. Every stage writes to a single audit ledger, so the full decision trail is reconstructable for any transaction at any time.

Stage	Function	Output
1 · Ingestion	Read a batch of failed payment and subscription records.	Normalized transaction objects
2 · Classifier	Classify the decline reason into one of six categories, using an LLM and rule-based checks.	Failure category label
3 · Policy engine	Map category to intervention and timing, enforcing retry caps and escalation rules.	Action and schedule

Stage	Function	Output
4 · Executor	Run the action against Razorpay test-mode APIs: retry the charge, generate an update-card link, or notify.	Execution result
5 · Ledger & reporting	Log every decision and outcome; aggregate into recovery-rate metrics and an exception list.	Audit trail and summary report

Failure categories and policy rules

Every failed transaction is classified into exactly one of six categories, each with a fixed, predetermined policy. This determinism is what makes the agent auditable — no decision is ad hoc.

Category	Policy
Insufficient funds	Retry once, timed three days out, aligned to a likely payday.
Card expired	No retry. Send a secure update-card link immediately.
Bank timeout or network error	Retry after one hour, up to two attempts.
Soft decline (do-not-honor)	Retry once after 24 hours; notify the customer if it fails again.
Fraud flag	Never auto-retried. Escalated immediately to a human reviewer.
Unresolved after three attempts	Marked unrecoverable and closed. No further action taken.

Technology stack

- **Backend** — Python with FastAPI, chosen for build speed and ease of live demonstration.
- **Reasoning layer** — an LLM API for decline-reason classification and for generating customer-facing recovery messages; this is the agentic core of the system.
- **Payments layer** — Razorpay test-mode APIs for payment links, retries, and subscription objects, tying the build directly to Razorpay's own infrastructure.
- **Data store** — SQLite or flat JSON for the transaction ledger, deliberately lightweight so build time goes into the decision loop rather than infrastructure.

- **Reporting** — a minimal dashboard showing recovery rate, amount recovered, and the exception list.

Meeting the stated bar

The brief asks specifically for a bounded, explainable, audited system. Each requirement is addressed directly:

- **Bounded** — a hard cap of three retry attempts per transaction; fraud-flagged transactions are never auto-retried under any condition.
- **Explainable** — every action logs its category, the policy rule that triggered it, and the outcome, so any decision can be traced back to a rule rather than a black box.
- **Audited** — a single append-only ledger records the full lifecycle of every transaction, so the submission can demonstrate its behavior rather than merely describe it.
- **Honestly reported** — the exception list, transactions the agent could not recover, is reported alongside the recovery rate, with no selective reporting of results.

Target demo result

The submission is built around a single, defensible statement, generated live from the agent's own ledger rather than scripted in advance:

> *“Of 62 failed payments totalling ■4.1 lakh, the agent recovered ■1.8 lakh — 44 percent — through automated retries and update-card nudges, correctly identified 9 transactions as unrecoverable, and escalated 3 fraud-flagged cases without touching them.”*

Prepared for the Razorpay AI Buildathon submission — Track 03: AI Revenue Recovery.